LA MÉTHODE DIVISER POUR RÉGNER

Une stratégie efficace

B. DARID

2026

SOMMAIRE

- 1 Introduction : L’union fait la force
- 2 Le triptyque de la méthode
- 3 Pourquoi est-ce si efficace ?
  - 3.1 Le saut de performance
- 4 Exemples d’application au programme
- 5 Sûreté et programmation
- 6 Défis rapides
  - 6.1 La stratégie du “Dico”
  - 6.2 L’arbre des appels
  - 6.3 Identification des étapes

1 Introduction : L'union fait la force

Face à un problème complexe (traiter des millions de données bancaires ou classer des archives électorales), la stratégie la plus efficace n'est pas de travailler plus vite, mais de **mieux organiser le travail**.

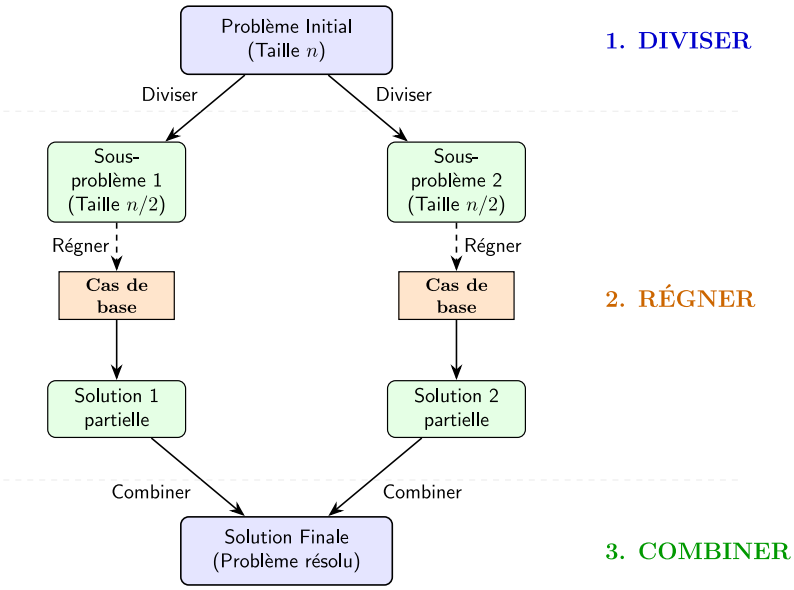
La méthode «Diviser pour régner» (*Divide and Conquer*) est un paradigme algorithmique qui consiste à décomposer un problème en sous-problèmes plus petits, jusqu'à ce qu'ils deviennent faciles à résoudre.

REPÈRE HISTORIQUE

En 1945, dans son texte fondateur First Draft of a Report on the EDVAC, le mathématicien John von Neumann formalise l'architecture des ordinateurs modernes. Puis, en 1948, décrit le tri fusion (Merge Sort). C'est la première application documentée de la stratégie «Diviser pour régner» sur une machine électronique.

John von Neumann

2 Le triptyque de la méthode



Diviser pour régner

Pour appliquer cette stratégie, on suit rigoureusement trois étapes:

- **DIVISER**: on découpe le problème initial en plusieurs sous-problèmes identiques mais de taille plus petite.

- **RÉGNER**: on résout les sous-problèmes. Si la taille est encore trop grande, on utilise la récursivité (on divise à nouveau). Si le problème est assez petit (cas de base), on le résout directement.
- **COMBINER**: on fusionne les solutions des sous-problèmes pour construire la solution du problème de départ.

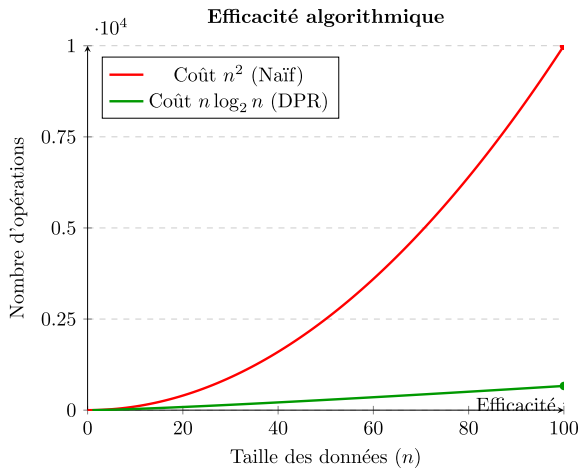
3 Pourquoi est-ce si efficace ?

En Terminale NSI, nous cherchons à réduire le coût d'exécution (en temps ou en mémoire).

3.1 Le saut de performance

L'intérêt majeur est de passer d'un coût quadratique (n^2) à un coût quasi-linéaire ($n \log_2 n$)

Taille des données (n)	Algorithme Naïf (n^2)	Diviser pour régner ($n \log_2 n$)	Rapport d'efficacité
1000	1000000	10000	x 100
1000000	1000000000000	20000000	x 50 000



Comparaison des performances

Note: le logarithme en base 2 ($\log_2 n$) représente ici le nombre de fois que l'on peut diviser n par 2 avant d'arriver à 1.

4 Exemples d'application au programme

Cette méthode se retrouve dans plusieurs domaines clés de notre cursus:

- le **tri fusion**: on divise une liste en deux, on trie chaque moitié et on les fusionne. C'est l'exemple type où l'étape de "combinaison" est cruciale.

- La **rotation d'image** (Bitmap): on divise l'image en quatre quadrants (haut-gauche, haut-droit, etc.) et on applique la rotation récursivement.
- La **recherche dichotomique**: bien qu'on ne traite qu'une seule moitié à chaque étape (ce qui simplifie la phase de combinaison), elle repose sur la division du problème pour atteindre un coût logarithmique.

5 Sûreté et programmation

Pour garantir un programme robuste et éviter les erreurs classiques (bugs):

- Condition d'arrêt: un algorithme récursif doit impérativement posséder un *cas de base* pour éviter que le débordement de la pile d'appels.
- Spécification: chaque fonction doit être documentée (entrées, sorties, préconditions) pour assurer la sûreté du code.

> **Activité à venir**
> Dans le prochain TP, nous mettrons en œuvre le Tri Fusion pour observer concrètement le gain de performance sur des listes de grande taille.

6 Défis rapides

6.1 La stratégie du "Dico"

- Vous cherchez le mot "Algorithme" dans un dictionnaire de 1000 pages.Si vous tournez les pages une par une, combien de pages devrez-vous examiner dans le pire des cas?
- Si vous utilisez la méthode "Diviser pour régner" (couper en deux à chaque étape), montrez qu'en seulement 10 étapes maximum, vous aurez trouvé la page.Donnée : $2^{10} = 1024$.

6.2 L'arbre des appels

On veut trier une liste de 8 éléments `[5, 2, 8, 3, 1, 9, 4, 6]` par tri fusion.

- Dessinez l'arbre de division de cette liste jusqu'à obtenir des listes de 1 élément.
- Combien de niveaux de division avez-vous obtenu ?
- Quel est le lien entre ce nombre de niveaux et $\log_2(8)$?

6.3 Identification des étapes

On considère l'algorithme de rotation d'une image de 4x4 pixels. On la divise en 4 carrés de 2x2.

- Quelle est l'étape de **division** ici?
- Que signifie l'étape **régner** pour un carré de 1x1 pixel?

- À votre avis, en quoi consiste l’étape de **combinaison** une fois que les petits carrés sont tournés?